
Quantum Shield

System Architecture Document

Quantum-Proof Systems Scanner — PNB Cybersecurity Hackathon

Team CyberRiot — June 17, 2026

Contents

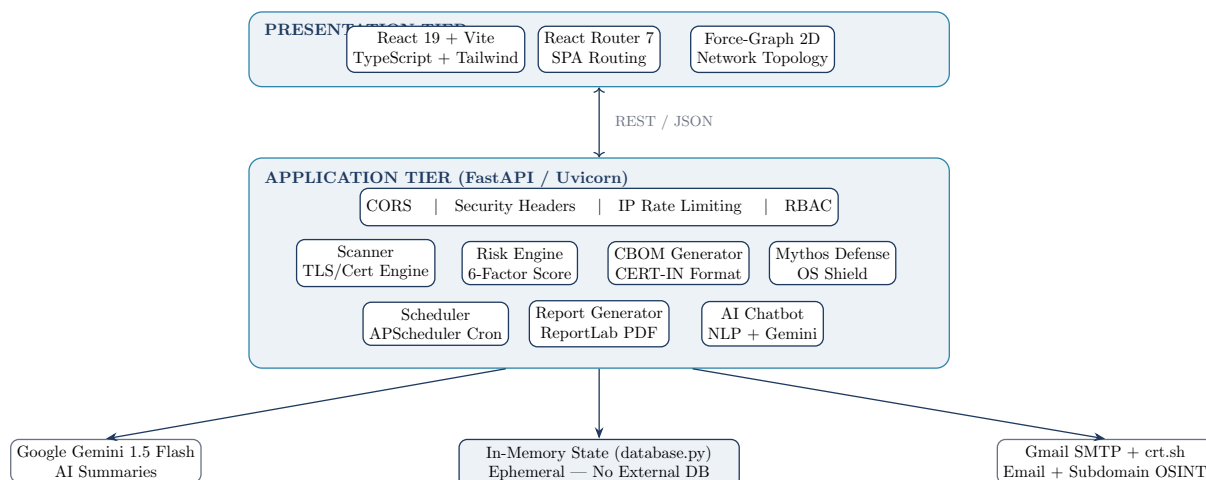
1	High-Level Overview	1
2	Architecture Diagram	2
3	Frontend — Presentation Tier	2
3.1	Stack	2
3.2	Auth and Session	2
3.3	Module Screens	2
4	Backend — Application Tier	3
4.1	Stack and Middleware	3
4.2	Engine Modules	3
4.2.1	Scanner (<code>scanner.py</code>)	3
4.2.2	Risk Engine (<code>risk_engine.py</code>)	3
4.2.3	CBOM Generator (<code>cbom_generator.py</code>)	3
4.2.4	Mythos OS Shield (<code>os_shield_engine.py</code>)	3
4.2.5	Scheduler (<code>scheduler.py</code>)	3
4.2.6	Report Generator (<code>report_generator.py</code>)	3
4.2.7	AI Chatbot (<code>chatbot.py</code>)	4
4.3	API Surface	4
5	Data Tier	4
6	External Integrations	4
7	Deployment	4
8	Production Considerations	4

High-Level Overview

Quantum Shield is a decoupled, two-tier client-server web application with a clear separation between a static front-end SPA and a stateful API engine. It assesses the quantum-cryptographic readiness of enterprise network assets by performing real TLS/certificate scanning, computing a multi-factor quantum risk score, generating Cryptographic Bills of Materials, and producing

executive PDF reports — all surfaced through an interactive dashboard with network topology visualization.

Architecture Diagram



Frontend — Presentation Tier

Stack

React 19 with Vite 8, TypeScript 5.9, Tailwind CSS 3.4, React Router 7 for client-side routing, and `react-force-graph-2d` for interactive network topology graphs. Deployed to Vercel as a static site with SPA rewrite rules.

Auth and Session

The app is auth-gated via `localStorage`. If no session exists, a login screen is rendered; otherwise the full console loads. Three roles are supported: **Super Admin**, **Admin**, and **User**.

Module Screens

All components live under `src/components/`:

- **Dashboard** — Central KPI overview with risk summaries
- **Asset Discovery / Inventory** — Subdomain enumeration and managed asset registry
- **Scanner** — TLS/certificate scan initiation and results
- **CBOM** — Cryptographic Bill of Materials viewer
- **PQC Posture** — Post-Quantum Cryptography readiness display
- **Cyber Rating** — Composite quantum risk rating
- **Mythos Defense** — Windows OS CVE shield module
- **Reports** — Executive PDF generation and download
- **AI Assistant** — Natural-language chatbot interface

All components use the `VITE_API_URL` env variable to reach the backend (defaults to port 8000).

Backend — Application Tier

Stack and Middleware

Python FastAPI served by Uvicorn (ASGI). Entry point: `backend/main.py`. Cross-cutting middleware includes CORS, custom security headers, per-IP per-bucket rate limiting, RBAC enforcement via `X-User-Role` header, and input validation with SSRF guards.

Engine Modules

Scanner (`scanner.py`)

The core scanning engine. Opens raw TCP sockets, wraps them with `ssl.SSLContext`, and probes each TLS version. Reads peer certificates, cipher suites, and key sizes using Python's `cryptography` library. Discovers subdomains via crt.sh certificate transparency logs and DNS brute-force. Optionally shells out to OpenSSL `s_client` for PQC/Kyber KEM detection. Uses `ThreadPoolExecutor` for concurrent scanning.

Risk Engine (`risk.engine.py`)

Computes a deterministic quantum risk score using a 6-factor weighted model:

Factor	Weight
Cryptographic strength	0.30
Protocol security	0.20
Known vulnerabilities	0.20
Exposure surface	0.10
Third-party risk	0.10
Governance posture	0.10

Applies expired-certificate caps, PQC-aware score floors, and classifies results into Low, Medium, High, or Critical.

CBOM Generator (`cbom_generator.py`)

Produces CERT-IN-style Cryptographic Bill of Materials in JSON — listing each algorithm, key size, OID, cipher suite, and a `pqc_safe` flag.

Mythos OS Shield (`os_shield.engine.py`)

Offline Windows vulnerability module. Maintains a CVE database (2024–2026), computes a patch-deficit score, and generates a remediation bundle (PDF report + JSON data + PowerShell hardening script) packaged as an in-memory ZIP download.

Scheduler (`scheduler.py`)

Recurring scan orchestrator using APScheduler with cron triggers. Parses natural-language scheduling expressions via `dateparser` and executes a full scan → score → PDF → email pipeline.

Report Generator (`report_generator.py`)

Builds executive PDF reports on-the-fly using ReportLab — includes KPI tables, bar/pie charts, and color-coded asset tables.

AI Chatbot (chatbot.py)

Regex and NLP-based intent parser supporting commands like SCAN, SCHEDULE_SCAN, and EMAIL_REPORT. Calls Google Gemini 1.5 Flash for AI-powered summaries (with graceful offline fallback) and dispatches emails via Gmail SMTP over STARTTLS.

API Surface

The backend exposes REST endpoints grouped as: Assets, Scanning, Scheduling, CBOM, Dashboard/Risk, Graph topology, Reports (with PDF downloads), Mythos OS Shield, User Governance, Auth/OTP, Email, and AI Chat — all under the `/api/` prefix.

Data Tier

There is no external database. `backend/database.py` holds all state in in-memory Python dicts and lists (`db_assets`, `db_jobs`, `db_nodes`, `db_edges`, `db_users`, `db_audit_logs`). Mock enterprise data is seeded on import. State is ephemeral — it resets on every restart and is not shared across Uvicorn workers.

External Integrations

Service	Purpose	Protocol
Google Gemini 1.5 Flash	AI-powered executive summaries	HTTPS
Gmail SMTP	Email report dispatch	STARTTLS (587)
crt.sh	Certificate transparency subdomain discovery	HTTPS
Scan Targets	TLS handshake and certificate inspection	TCP 443

Deployment

The frontend is built with `tsc -b && vite build` and deployed to Vercel as a static site. The backend runs independently via `uvicorn main:app` on port 8000 (dev) or behind a reverse proxy on 443 (production). The two are connected via the `VITE_API_URL` build-time variable.

Live demo: <https://cyberriot-quantum-shield.vercel.app/>

Production Considerations

1. Replace in-memory storage with PostgreSQL or Redis for persistence.
2. Tighten CORS to the specific frontend domain (currently wildcard for demo).
3. Move credentials to a secrets manager; remove `.env` from version control.
4. Govern outbound scanning per organizational network/security policy.